МОНГОЛ УЛСЫН ИХ СУРГУУЛЬ
МЭДЭЭЛЛИЙН ТЕХНОЛОГИ, ЭЛЕКТРОНИКИЙН СУРГУУЛЬ
МЭДЭЭЛЭЛ, КОМПЬЮТЕРИЙН УХААНЫ ТЭНХИМ

Тооцоолол ба өгөгдлийн ухаан

#4 - Python программчлал

(list comprehension, function obj., testing, debugging)

Өнгөрсөн хичээлээр

- Tuple

```
( 1, "two", 3, (4, 5), [6,] )
```

- List

```
[ 1, "two", 3, [4, 5], (6,) ]
```

- Dictionary

```
{ 1: "two", 3: [4, 5], "six": 7 }
```

LIST COMPREHENSIONS

List comprehensions

- Жагсаалтын **бүх элементэд үйлдэл гүйцэтгэж**, шинэ жагсаалт үүсгэх нь үйлдэл өргөн хэрэглэгддэг

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

шинэ жагсаалт

гүйцэтгэх үйлдэл

List comprehensions

- Python хэлэнд үүнийг товчоор, нэг мөрөнд бичих боломж байдаг бөгөөд **list comprehension** гэнэ.
  - Шинэ жагсаалт үүсгэнэ
  - Жагсаалтын элемент бүрт функц хэрэглэнэ
  - (Optional) нөхцөл хангасан элементээр шүүж болно

```
[expression for elem in iterable if test]
```

List comprehension - Бүх элемент

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

```
Lnew = [ e**2 for e in L ]
```

List comprehension - Бүх элемент

шинэ жагсаалт

бүх элементээр гүйх

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

```
Lnew = [ e**2 for e in L ]
```

гүйцэтгэх
функц

List comprehension - Нөхцөл шалгах

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

```
def f(L):
    Lnew = []
    for e in L:
        if e%2 == 0:
            Lnew.append(e**2)
    return Lnew
```

зөвхөн энэ нөхцлийг хангах утгыг боловсруулж, шинэ жагсаалтад нэмэх

```
Lnew = [ e**2 for e in L ]
```

List comprehension - Нөхцөл шалгах

```
def f(L):
    Lnew = []
    for e in L:
        Lnew.append(e**2)
    return Lnew
```

```
def f(L):
    Lnew = []
    for e in L:
        if e%2 == 0:
            Lnew.append(e**2)
    return Lnew
```

зөвхөн энэ нөхцлийг хангах утгыг боловсруулж, шинэ жагсаалтад нэмэх

```
Lnew = [ e**2 for e in L ]
```

```
Lnew = [ e**2 for e in L if e%2 == 0 ]
```

List comprehensions

```
[expression for elem in iterable if test]
```

- Жагсаалтын бүх элементээр гүйж, **нөхцөл хангах утгуудыг боловсруулж**, **шинэ жагсаалт** үүсгэнэ.

```
[e**2 for e in range(6)]                    → [0, 1, 4, 9, 16, 25]
[e**2 for e in range(8) if e%2 == 0]        → [0, 4, 16, 36]
[[e,e**2] for e in range(4) if e%2 != 0]    → [[1,1], [3,9]]
```

Туршиж үзэцгээе

- Доорх илэрхийллээс ямар утга буцах вэ?
  - Алхам 1: Дараалалд ямар утгууд байна вэ?
  - Алхам 2: Дарааллаас ямар утгууд шүүгдэж үлдэх вэ?
  - Алхам 3: Үлдсэн утгууд дээр үйлдлийг гүйцэтгэх.

```
[len(x) for x in ['xy', 'abcd', 7, '4.0'] if type(x) == str]
```

Функцийн өгөгдмөл параметр

Язгуурыг хэсэглэх аргаар хайх

```
def bisection_root(x):
    epsilon = 0.01
    low = 0
    high = x
    guess = (high + low)/2.0
    while abs(guess**2 - x) >= epsilon:
        if guess**2 < x:
            low = guess
        else:
            high = guess
        guess = (high + low)/2.0
    return guess

print(bisection_root(123))
```

Нэмэлт параменр

- Шаардлага: Илүү нарийн хариу хэрэгтэй үед `def bisection_root(x)`-г яаж сайжруулах вэ?
- Боломж:
  - epsilon-г функцийн **дотор өөрчлөх** (хэрэглэж буй бүх газарт нөлөөлнө)
  - epsilon-г функцийн **гадна зарлах** (global хувьсагч хэрэглэх нь муу туршлага)
  - epsilon-г функцийн **аргумент** болгох

epsilon параметр

```
def bisection_root(x, epsilon):
    low = 0
    high = x
    guess = (high + low)/2.0
    while abs(guess**2 - x) >= epsilon:
        if guess**2 < x:
            low = guess
        else:
            high = guess
        guess = (high + low)/2.0
    return guess

print(bisection_root(123, 0.01))
```

Түлхүүр үгт параметр, өгөгдмөл утга

`def bisection_root(x, epsilon)`-г цааш нь дахин сайжруулж болно

- epsilon-г аргумент болгон функц рүү дамжуулж өгсөн
  - **Ихэнх тохиолдолд** бидэнд **стандарт утга** 0.01 байхад хангалттай
  - **Зарим тохиолдолд өөр утга** хэрэглэх шаардлага гарна
- Түлхүүр үгт параметр (keyword parameter) буюу **өгөгдмөл параметр** (**default parameter**) хэрэглэх болно

epsilon keyword parameter

```
def bisection_root(x, epsilon=0.01):
    low = 0
    high = x
    guess = (high + low)/2.0
    while abs(guess**2 - x) >= epsilon:
        if guess**2 < x:
            low = guess
        else:
            high = guess
        guess = (high + low)/2.0
    return guess

print(bisection_root(123))
print(bisection_root(123, 0.05))
```

0.01 өгөгдмөл утгатай, default parameter

epsilon-г зааж өгөгөөгүй учраас өгөгдмөл утга 0.01-р тооцоолно

epsilon-г 0.5-р тооцоолно

Түлхүүр үгт параметрийн дүрэм

- **Функцийн тодорхойлолтод**:
  - Өгөгдмөл параметрүүд үргэлж төгсгөлд байна
- Функц дуудахад дараах тохиолдлууд **зөвшөөрөгдөнө**:
  - `bisection_root_new(123)`
  - `bisection_root_new(123, 0.001)`
  - `bisection_root_new(123, epsilon=0.001)`
  - `bisection_root_new(x=123, epsilon=0.1)`
  - `bisection_root_new(epsilon=0.1, x=123)`
- Функц дуудахад дараах тохиолдууд **болохгүй**:
  - `bisection_root_new(epsilon=0.001, 123) #error`
  - `bisection_root_new(0.001, 123) #no error but wrong`

Функц объект

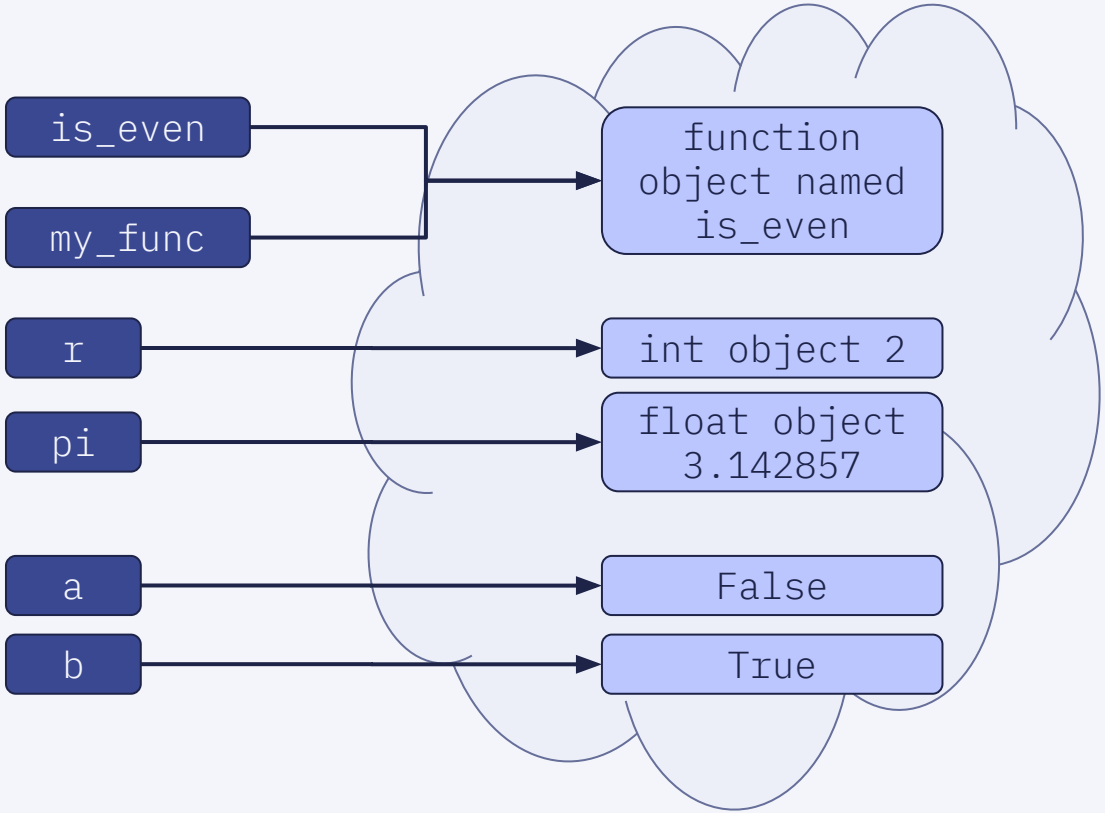
Программын объект

```
def is_even(i):
    return i%2 == 0

r = 2
pi = 22/7
my_func = is_even
a = is_even(3)
b = my_func(4)
```

функц дуудаагүй, зөвхөн нэр

функц дуудсан



Функц, функц буцааж болно

```
def make_prod(a):
    def g(b):
        return a*b
    return g
```

функц дотор функц хэрэгжүүлсэн

функц дуудаагүй

```
val = make_prod(2)(3)
print(val)
```

АДИЛ

```
doubler = make_prod(2)
val = doubler(3)
print(val)
```

Функц объект

- Программын загварчлалд хэрэг болно
  - **Задлах**, **хийсвэрлэх** санаа
  - Кодоо бүтэцлэх өөр нэгэн **хэрэгсэл**
- Гүйцэтгэлийг таслах
  - **Control-flow**-н нэг жишээ
  - **Partial execution**

Testing & Debugging

Согог (Bug)

Компьютер, программ хангамж гарахаас өмнө XIX-р зууны үеэс тоног төхөөрөмжийн ямар нэг **согог**ийг англиар “**bug**” хэмээн нэрлэж иржээ.

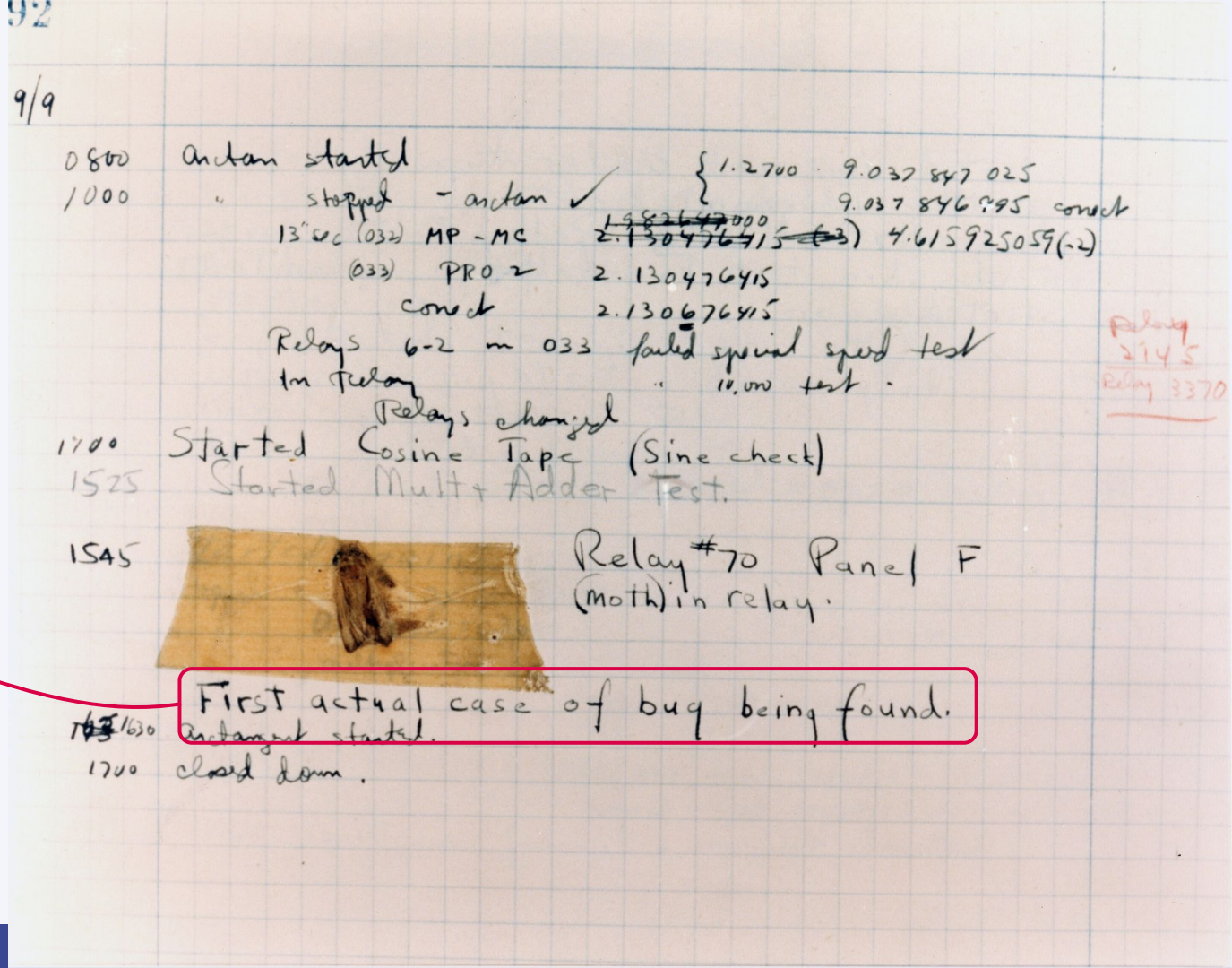
- **Алдааны төрөл:**
  - Логикийн алдаа (Logic/Semantic error)
  - Бичиглэл, статик утга зүйн алдаа (Syntax, static semantics error)
- Testing - Алдаа олох, шалгах
- Debugging - Алдааг судлах, засах

Компьютерийн анхны “жинхэнэ” bug

1947 онд Грэйс Хоппер Харвардад Марк II компьютер дээр ажиллаж байв.

Нэг өдөр программд нь гарсан алдааг шалгаж, явж байгаад компьютерийн нэг эд ангиас дэнгийн эрвээхэй олжээ.

"First actual case of bug being found."



DEFENSIVE PROGRAMMING

- Функцийн **тодорхойлолт** бичих
- Прогарммаа **модульчлах**
- Оролт/гаралтыг **шалгах** (assertions)

TESTING

- Оролт/Гаралтын хослолыг тодорхойлолттой таарч байгаа эсэхийг **шалгах**
- “Энэ ажиллахгүй байна!”
- “Программаа хэрхэн эвдэх вэ?”

DEBUGGING

- Алдаа үүсгэж буй тохиолдлуудыг **судлах**
- “Яагаад ажиллахгүй байна?”
- “Программаа хэрхэн засах вэ?”

Шалгах, дебаг хийхэд хялбар байхаар зохиомжил

- Программ хөгжүүлэлтийн **эхэн үеэс** шалгах, дебаг хийхэд хялбар байхаар загварчил
- Программаа бие даасан, тус бүрийг нь шалгах боломжтой **модуль** болгон хуваах
- Модулийн **шаардлагыг бичиж, тэмдэглэх**
  - Оролт нь ямар байх ёстой?
  - Гаралт нь ямар байх ёстой?
- Кодын бүтцийн **төсөөллөө бичиж тэмдэглэх**

Хэзээ шалгахад бэлэн болох вэ?

- **Код ажиллаж байх**
  - Syntax error засах
  - Static semantic error засах
    - Python interpreter эдгээрийг ихэнх тохиолдолд олоод өгнө
- **Шалгалтын өгөгдөлтэй байх**
  - Оролтын өгөгдөл
  - Оролтын өгөгдөлд харгалзах гаралтын өгөгдөл

Шалгалтын төрлүүд

- **Unit testing**
  - Программын хэсэг бүрийг шалгана
  - **Функц бүрийг** тусад нь шалгана
- **Regression testing**
  - Алдаа олох бүрдээ тухайн алдааг шалгах шалгалт нэмэх
  - Өмнө нь зассан **алдаа дахин гарч ирж байгаа эсэх**ийг олж мэдэх
- **Integration testing**
  - Программ нийтдээ ажиллаж байгаа эсэх?
  - Хүмүүс ихэвчлэн шууд энэ шалгалтыг хийх гэж оролддог

Шалгалтын аргууд

- Бодлогод гарч болох **заагуудыг (natural boundaries) тунгаах**

```
def is_bigger(x, y):
    """ Assumes x and y are ints
    Returns True if y is less than x, else False """
```

- Зааг олдохгүй бол **санамсаргүй (random) тохиолдлуудаар шалгах**
  - Код алдаагүй байх эсэх магадлал олон шалгалт хийх тусам өснө
  - Доорх хоёр арга үүнийг бодвол илүү боловсронгуй
- **Хар хайрцагны (black box) шалгалт**
  - Тодорхойлолтын дагуу шинжих
- **Шилэн хайрцагны (glass box) шалгалт**
  - Кодын дагуу шинжих

Black box testing

```
def sqrt(x, eps):
    """ Assumes x, eps floats, x >= 0, eps > 0
    Returns res such that x-eps <= res*res <= x+eps """
```

- Программын кодыг **харах шаардлагагүй**
- Хэрэгжүүлсэн хүнээс өөр хүн шалгах боломжтой, **bias** байхгүй
- Хэрэгжүүлэлт өөрчлөгдсөн ч шалгалтыг **дахин явуулж болно**
- Тодорхойлолтоос шалгах **зам**уудыг тодорхойлно
  - Бодлогын онцгой (зааг) хэсгүүдэд шалгалт хийнэ
  - Boundary нөхцлийг шалгах (хоосон жагсаалт, нэг элементтэй жагсаалт, том тоо, бага тоо)

Black box testing

```
def sqrt(x, eps):
    """ Assumes x, eps floats, x >= 0, eps > 0
    Returns res such that x-eps <= res*res <= x+eps """
```

тохиолдол	x	eps
зааг	0	0.0001
бүхэл тоон квадрат	25	0.0001
нэгээс бага	0.05	0.0001
иррационал	2	0.0001
экстрим	2	1.0 / 2.0 ** 64.0
экстрим	1.0 / 2.0 ** 64.0	1.0 / 2.0 ** 64.0
экстрим	2.0 ** 64.0	1.0 / 2.0 ** 64.0
экстрим	1.0 / 2.0 ** 64.0	2.0 ** 64.0

Glass box testing

- **Программын кодоос** шалгалт бичих чиглэл тодорхойлно
- Кодын бүх хэсэг ядаж нэг удаа ажиллах шалгалтууд хийгдсэн бол **path-complete** гэнэ
- Сул тал?
  - Давталтаар тодорхой тооны давталт л гүйцэтгэнэ
  - Тохиолдол хаягдах эрсдэлтэй
- Аргачлал:
  - Салбарлах: Бүх нөхцлийг шалгаж, ажиллуулах
  - For давталт:
    - давталт руу ороогүй
    - нэг удаа давтсан
    - нэгээс олон давтсан
  - While давталт: for давталттай адил, нэмээд бүх гарах нөхцлийг шалгах

Glass box testing

```
def abs(x):
    """ Assumes x is an int
    Returns x if x>=0 and -x otherwise """
    if x < -1:
        return -x
    else:
        return x
```

- Path-complete шалгалтаас **алдаа олохгүй** өнгөрөх боломжтой
- Path-complete шалгалтын тохиолдол: 2 болон -2
- Гэвч abs(-1) үед -1 гэж буруу утга буцаана
- Онцгой хэсэг буюу зааг хэсгийн тохиолдлуудыг шалгах шаардлагатай

Debugging

- Программын код зорьсноос буруу ажиллаж байгааг мэдсэний дараа:
  - Алдааг тусгаарлах
  - Алдааг засах
  - Бүх тохиолдолд зөв ажилладаг болох хүртэл шалгах
- Зорилго: Алдаагүй программ бичих
- Багаж:
  - IDLE, Anaconda гэх мэт хэрэгсэлд хамт ирдэг
  - **Python Tutor**
  - **print** statement
  - Тархиа ажиллуулах, ямар учраас алдаа гарсныг **систем**тэйгээр хайх

Алдааны мессеж - Хялбар

- Жагсаалтын хэмжээнээс гадна элемент рүү хандах

```
test = [1, 2, 3]
test[4]                                  → IndexError
```

- Тохиромжгүй төрөл хооронд хувиргах

```
int(test)                                → TypeError
```

- Байхгүй хувьсагч руу хандах

```
a                                        → NameError
```

- Төрөл хувиргалгүй өөр төрлийн утгуудыг хольж хэрэглэх

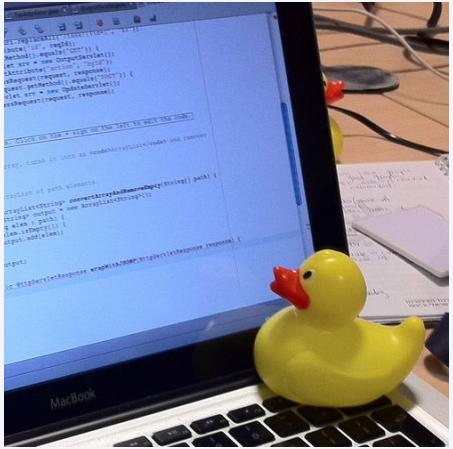
```
'3' / 4                                  → TypeError
```

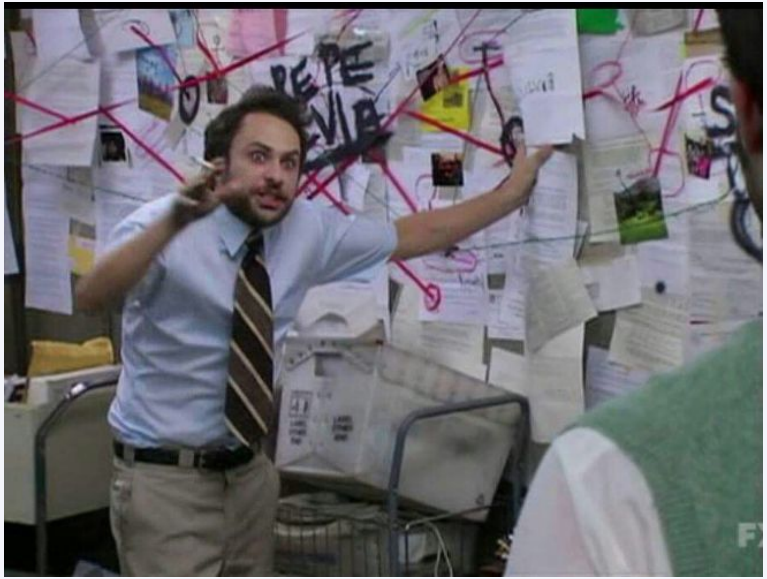
- Хаалт, хашилт зэргийг хаахаа мартах

```
a = len([1, 2, 3]
print(a)                                 → SyntaxError
```

Логикийн алдаа - Хүнд

- Шинэ код бичихээсээ өмнө **бодох**
- Зураг **зурах**, завсарлага авах
- Кодоо **тайлбарлах**
  - Өөр хүнд
  - Rubber duck (debugging)





Debugging Steps

- Программын кодыг **судлах**
  - Юу бурууг бүү асуу
  - Яагаад буруу хариу гарсныг асуу
- **Шинжлэх ухааны** арга
  - Өгөгдлийг судлах
  - Таамаг дэвшүүл
  - Туршилтыг давт
  - Хамгийн хялбар оролтоор шалга

Print statements

- **Таамгаа шалгахад** тохирсон арга
- Хэзээ хэвлэх вэ?
  - Функц рүү орох үед
  - Параметрүүдийг шалгах
  - Үр дүн шалгах
- **Хоёртын хэсэглэх арга** хэрэглэх
  - Кодын дунд хэвлэх
  - Алдаа аль хэсэгт гарч байгааг хайж олох

```
def some_function(a, b):
    print("start", a, b)
    #
    # some logic...
    #
    print("working!", foo)
    #
    # some logic...
    #
    result = ...
    print("done", result)
    return result
```